

CIS 433 — Project 2

Smile Manipulation via Latent Space Traversal in a VAE

Muhammad Hadi Hussain | Simon Business School, University of Rochester | April 2025

1. Project Overview

This project extends Project 1 by applying a Variational Autoencoder (VAE) to the CelebA celebrity face dataset and using the learned 128-dimensional latent space to perform semantic face editing. The specific task is to find the "smile direction" in latent space and use it to make any face look more or less smiley, without directly manipulating any pixel.

Unlike Project 1, which used a 2D latent space on MNIST digits that could be plotted and explored visually, the 128-dimensional CelebA space cannot be directly visualized. Instead, the approach relies on vector arithmetic: computing attribute-conditional means across thousands of faces to isolate the latent direction that encodes smiling, then traversing along that direction at inference time.

The instructor provided a pretrained model checkpoint to allow the project to focus on latent space analysis rather than the computationally expensive training step. Per the course guidance: "The key part of the second project is not about training the model itself — it is about using the model to travel in the latent space."

2. Model Architecture

The VAE is composed of three PyTorch modules — an Encoder, a Sampler, and a Decoder — all operating on 128×128 RGB images with a 128-dimensional latent space.

Encoder

The encoder compresses an input image into a latent distribution. It applies five strided convolutional layers (kernel size 3, stride 2, padding 1), each followed by Batch Normalization and Leaky ReLU activation. The five layers progressively halve the spatial dimensions: 128→64→32→16→8→4. The resulting 128×4×4 feature map is flattened into a 2048-element vector, which feeds two independent linear heads — one outputting the latent mean (μ) and the other outputting the log-variance.

Sampler

The sampler implements the reparameterization trick, which allows gradients to flow through the stochastic sampling step during training. Given μ and $\log_var$ from the encoder, it draws a latent sample:

$$z = \mu + \exp(0.5 \cdot \log_var) \cdot \varepsilon, \quad \varepsilon \sim N(0, I)$$

At inference time, only μ is used — the variance branch is dropped — so that edits are deterministic and identity-preserving.

Decoder

The decoder mirrors the encoder in reverse. A linear layer projects the 128-dim latent vector back to a 128×4×4 feature map (with Batch Normalization and Leaky ReLU), then five transposed convolutional layers (ConvTranspose2d, stride 2) progressively upsample back to 128×128. A final transposed convolution with Sigmoid activation produces the output RGB image.

Training Objective

The VAE is trained to minimize a combined loss: pixel-wise MSE reconstruction loss (faithfulness) plus a KL divergence penalty that regularizes the latent space to follow a standard Gaussian prior. The pretrained weights provided by the instructor (vae_celeba_128x128_128.pt) were used for all experiments. An optional training cell is included in the notebook to demonstrate the full pipeline with a configurable beta weight on the KL term.

3. Data Preparation

The CelebA aligned face dataset (img_align_celeba.zip) originally used in Project 1 was reused. The key preparation steps were:

- Images resized to 128×128 using torchvision.transforms.Resize and converted to float tensors in [0, 1].
- Attribute labels loaded from list_attr_celeba.csv. The Smiling column uses {-1, +1} encoding in the raw file; this was normalized to {0, 1} before use.
- A custom CelebASmilingDataset class (PyTorch Dataset) loads images on-the-fly from disk and returns the image tensor along with the Smiling label for each sample.
- DataLoader configured with batch size 256 and shuffle=False (order must be consistent for convergence tracking). The num_workers parameter was set to 2; on Windows, setting this to 0 avoids multiprocessing issues as flagged in the course materials.
- The zip archive was extracted to local Colab storage (/content/) rather than read directly from Google Drive, significantly reducing I/O bottleneck over 200,000 images.

4. Finding the Smile Direction

The central insight is that in a well-trained VAE, semantically meaningful attributes tend to correspond to directions in latent space. To find the smile direction, the approach computes the average latent encoding of smiling faces minus the average latent encoding of non-smiling faces:

```
smile_vector = mean{ mu(x) : Smiling=1 } - mean{ mu(x) : Smiling=0 }
```

Rather than making two full passes through the dataset, an online convergence algorithm is used. After each batch, the running class means are updated and their L2 delta (change since the previous batch) is computed. When the combined delta for both classes falls below a threshold of 0.01, the means are considered converged and the loop exits early. The final smile vector is then normalized to unit length before use.

This approach, based on the instructor's helper code, is efficient and robust: it exploits the fact that means converge quickly with large batches, and the convergence check prevents unnecessary computation once the direction has stabilized.

5. Latent Space Manipulation

Once the smile vector is found, editing any face reduces to simple vector arithmetic in latent space. For a given input image, the encoder produces μ (the mean of the posterior distribution). The smile intensity is then controlled by a scalar α :

```
z_more_smiley = mu + alpha * smile_vector  
z_less_smiley = mu - alpha * smile_vector
```

The modified latent vectors are decoded back to image space using the decoder. An α value of 3.0 was selected for the main experiments, producing clearly visible but realistic smile edits without distorting facial identity.

6. Results

Reconstruction Quality

A random CelebA image was encoded and reconstructed (decoded directly from μ , no noise). The reconstruction faithfully captured face structure, skin tone, hair color, and expression with mild softening of fine details — a known characteristic of VAE reconstruction due to the MSE loss averaging over pixel uncertainty.

Smile Edits on Five Test Images

Five test images (000007, 000025, 000100, 000500, 001000) were each shown in four panels: original, reconstruction, less smiley ($\alpha=-3$), and more smiley ($\alpha=+3$). Across all five faces, the edits produced consistent and plausible results:

- Faces with neutral or sad expressions gained visible upward mouth curvature and subtle cheek lift when edited toward more smiley.
- The reverse direction produced downturned or flat expressions.
- Non-smile features — skin tone, eye shape, hair, background, face identity — remained largely stable, confirming that the smile vector is disentangled from other attributes.

Continuous Alpha Sweep

A sweep over alpha values $\{-4, -2, 0, +2, +4\}$ on image 000025.jpg demonstrated the continuous and monotonic nature of the smile direction. Progressing from strongly negative to strongly positive alpha produced a smooth gradient of expression — from visibly unhappy to broadly smiling — with no discontinuities or artifacts, consistent with a well-regularized latent space.

7. Conclusion

This project demonstrated that a pretrained VAE can serve as a powerful engine for semantic face editing entirely in latent space. By identifying the smile attribute direction through class-conditional mean subtraction and traversing along it with a simple scalar parameter, smooth and identity-preserving smile edits were achieved across diverse faces without any direct image manipulation.

The results highlight a fundamental property of well-trained generative models: meaningful attributes are encoded as structured directions in the latent space. This principle underlies many real-world applications in generative AI, including face aging, style transfer, and expression synthesis.

8. Submitted Files

- Hadi_Hussain_CIS433_Project2.ipynb — Main notebook with all code, outputs, and inline narrative
- hadi_project2_model_submission.pt — Pretrained VAE state dict (verified with reload check)